IY4101 – Object Oriented Programming
Practical Programming Assignment 2

Report

Michael Tahiroğlu
304000037

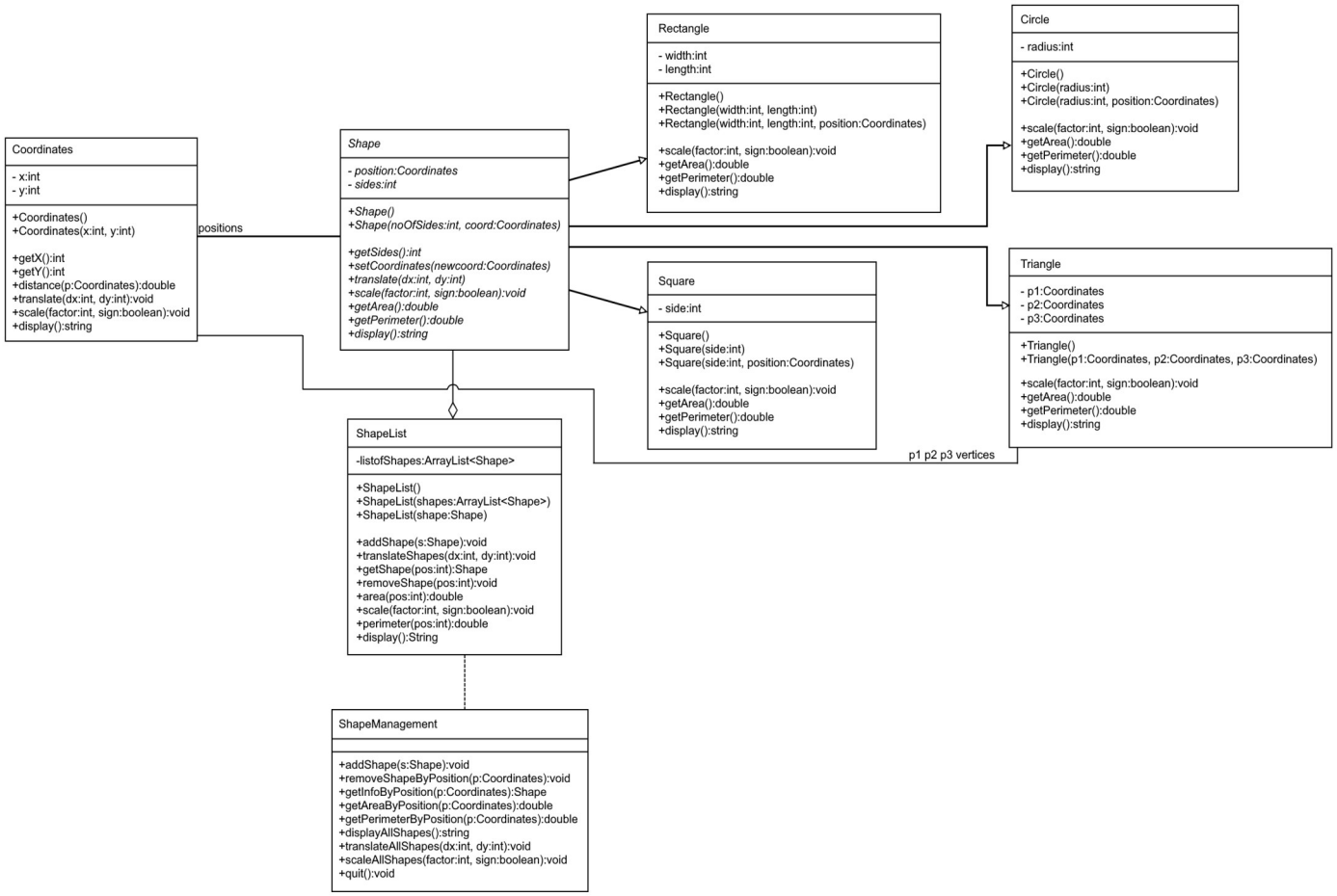
I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.

Design

The shape management tool has three main classes: Shape, ShapeList, and ShapeManagement. Shape contains four sub-classes that define the supported types of shapes: Rectangle, Square, Circle and Triangle. All of these shapes can be defined, translated, and scaled. The area and perimeter can also be calculated. This information can also be displayed.

ShapeList contains an ArrayList of Shape instances. It supports commands to translate, scale, and display all Shape instances within it. ShapeManagement is the main class that creates a console interface for the user to create and view their shapes in.

Below is the UML Diagram for the tool:



Testing

The required tests were run by applying the specified tests with the ShapeManagement class. Another common type of Shape was a unit shape (such as a triangle with vertices at (0, 0), (0, 1), (1, 0)). Below are the results of this test:

- Creating a triangle whose vertices are at (50, 50), (20, 70) and (70, 70), adding it to the list.
    - **Expectation:** The triangle will have an area of 500 and a perimeter of around 114.34. It will store it as the 0th indexed element of the ShapeList.
    - **Results:** The triangle's coordinates show properly, the areas are indeed correct. It is the 0th indexed element of the ShapeList.

```
------
Triangle. Vertices:
        X = 50, Y = 50
        X = 20, Y = 70
        X = 70, Y = 70
Area: 500.0
Perimeter: 114.3397840021018
------
```

- Creating a rectangle at (100, 20), with width 10, length 15, adding it to the list.
    - **Expectation:** The rectangle is created at (100, 20). Has an area of 150 and a perimeter of 50. It should be the 1st indexed element of the ShapeList.
    - **Results:** The rectangle is at (100, 20). It has an area of 150 and a perimeter of 50, as expected. It is the 1st indexed element of the ShapeList.

```
------
Rectangle at: X = 100, Y = 20
Width: 10
Length: 15
Area: 150.0
Perimeter: 50.0
------
```

- Creating a circle in position (80, 100) and a radius of 25. Adding it to the list.
    - **Expectation:** A circle is created at (80, 100), with the radius set to 25. It should have an area around 1963.495, and a perimeter around 157.08. It should be the 2nd indexed element of the ShapeList.
    - **Results:** The circle is at the correct location with the correct radius. It has the correct area and perimeter.

```
------
Circle at: X = 80, Y = 100
Radius: 25
Area: 1963.4954084936207
Perimeter157.07963267948966
------
```

- Creating a square at (90, 40) with a side length of 20, adding it to the list.
    - **Expectation:** Square is created, with an area of 400 and a perimeter of 80. It should be the 3rd indexed element of ShapeList.
    - **Results:** The square is at the correct location and has the correct perimeter and area.

```
------
Square at: X = 90, Y = 40
Side length: 20
Area: 400.0
Perimeter: 80.0
------
```

- Adding three other shapes: A circle at (0, 0) with a radius of 10 (4th shape), A triangle with points (0, 0), (0, 1), (1, 0) (5th shape), and a rectangle at (4, 4) with width 2 and length 4 (6th shape)
    - **Expectation:** The shapes are added and present at the correct index.
    - **Results:**

```
------
Circle at: X = 0, Y = 0
Radius: 10
Area: 314.1592653589793
Perimeter62.83185307179586
------
```

```
------
Triangle. Vertices:
        X = 0, Y = 0
        X = 0, Y = 1
        X = 1, Y = 0
Area: 0.49999999999999983
Perimeter: 3.414213562373095
------
```

```
------
Rectangle at: X = 4, Y = 4
Width: 2
Length: 4
Area: 8.0
Perimeter: 12.0
------
```

- Show the area and perimeter of the second object in the list.
    - **Expectation:** The second shape is the 1st index, so putting in 1 in there will give the rectangle at (100, 20), with width 10, length 15. The area then is 150 and the perimeter is 50.
    - **Results:** The area and perimeter are correct.

```
Area: 150.0
Perimeter: 50.0
```

- Display information about all the shapes.
    - **Expectation:** The result will show each shape that we've seen so far in the order we've added them.
    - **Results:** Indeed the shapes appear in the correct order.

```
-------
Triangle. Vertices:
        X = 50, Y = 50
        X = 20, Y = 70
        X = 70, Y = 70
Area: 500.0
Perimeter: 114.3397840021018
------

-------
Rectangle at: X = 100, Y = 20
Width: 10
Length: 15
Area: 150.0
Perimeter: 50.0
------

-------
Circle at: X = 80, Y = 100
Radius: 25
Area: 1963.4954084936207
Perimeter157.07963267948966
------
```

```
-------
Square at: X = 90, Y = 40
Side length: 20
Area: 400.0
Perimeter: 80.0
------

-------
Circle at: X = 0, Y = 0
Radius: 10
Area: 314.1592653589793
Perimeter62.83185307179586
------
```

```
-------
Triangle. Vertices:
        X = 0, Y = 0
        X = 0, Y = 1
        X = 1, Y = 0
Area: 0.49999999999999983
Perimeter: 3.414213562373095
------

-------
Rectangle at: X = 4, Y = 4
Width: 2
Length: 4
Area: 8.0
Perimeter: 12.0
------
```

- Remove the third shape and check that no error occurs.
  - **Expectation:** The third shape, the circle at (80, 100) will get removed. Therefore when we check the third position, we should find our current fourth shape, the square at (90, 40) in its place. Keep in mind the third shape is at the 2nd index.
  - **Result:**

```
Enter shape position in list:
2

------
Square at: X = 90, Y = 40
Side length: 20
Area: 400.0
Perimeter: 80.0
------
```

- Translate all the shapes by 10 for coordinate x and 15 for coordinate y.
  - **Expectation:** The X coordinate for the 4th shape, the circle at (0, 0) would be (10, 15) now.
  - **Result:** This is indeed the case.

```
------
Circle at: X = 10, Y = 15
Radius: 10
Area: 314.1592653589793
Perimeter62.83185307179586
------
```

- Display information about all the shapes.
  - **Expectation:** We should see all the shapes, with their coordinates shifted accordingly.
  - **Result:**

```
-------
Triangle. Vertices:
        X = 60, Y = 65
        X = 30, Y = 85
        X = 80, Y = 85
Area: 500.0
Perimeter: 114.3397840021018
------

-------
Rectangle at: X = 110, Y = 35
Width: 10
Length: 15
Area: 150.0
Perimeter: 50.0
------

-------
Square at: X = 100, Y = 55
Side length: 20
Area: 400.0
Perimeter: 80.0
------
```

```
-------
Circle at: X = 10, Y = 15
Radius: 10
Area: 314.1592653589793
Perimeter62.83185307179586
------

-------
Triangle. Vertices:
        X = 10, Y = 15
        X = 10, Y = 16
        X = 11, Y = 15
Area: 0.49999999999999983
Perimeter: 3.414213562373095
------

-------
Rectangle at: X = 14, Y = 19
Width: 2
Length: 4
Area: 8.0
Perimeter: 12.0
------
```

- Increase all the shapes by a factor of 2
    - **Expectation:** We should see the test triangle with points (10, 15), (10, 16), (11, 15) turn into the points (20, 30), (20, 32), (22, 30).
    - **Result:**

```
------
Triangle. Vertices:
        X = 20, Y = 30
        X = 20, Y = 32
        X = 22, Y = 30
Area: 1.9999999999999993
Perimeter: 6.82842712474619
------
```

- Display all the shapes with the translation and scaling.
    - **Expectations:** The shapes should all be there, except for the one we removed, and they should all have the appropriate scaling.
    - **Result:**

```
-------
Triangle. Vertices:
        X = 120, Y = 130
        X = 60, Y = 170
        X = 160, Y = 170
Area: 2000.0
Perimeter: 228.6795680042036
------

-------
Rectangle at: X = 220, Y = 70
Width: 20
Length: 30
Area: 600.0
Perimeter: 100.0
------
```

```
-------
Square at: X = 200, Y = 110
Side length: 40
Area: 1600.0
Perimeter: 160.0
------

-------
Circle at: X = 20, Y = 30
Radius: 20
Area: 1256.6370614359173
Perimeter125.66370614359172
------
```

```
-------
Triangle. Vertices:
        X = 20, Y = 30
        X = 20, Y = 32
        X = 22, Y = 30
Area: 1.9999999999999993
Perimeter: 6.82842712474619
------

-------
Rectangle at: X = 28, Y = 38
Width: 4
Length: 8
Area: 32.0
Perimeter: 24.0
------
```

Further testing was also done to see if the program can handle common errors (such as an invalid index, trying to divide the scale by zero, or an invalid command number)

- Entering an invalid index to remove a shape from
    - **Expectations:** If I try to remove a shape at index 20, the program should let us know.
    - **Result:** it tells us by displaying “Invalid index.”

```
2
Enter shape position in list:
20
Invalid index.
```

- Entering an invalid index to get information about a shape.
    - **Expectations:** The program should display the same error.
    - **Result:** It does.

```
3
Enter shape position in list:
20
Invalid index.
```

- Entering an invalid index to get area and perimeter.
    - **Expectations:** The program should display the same error again.
    - **Result:**

```
4
Enter shape position in list:
20
Invalid index.
```

- Trying to divide the scale by zero.
  - **Expectations:** The program should raise a division by zero error, catch it, and display it.
  - **Result:**

```
7
Divide scale or multiply? (0 for divide, 1 for multiply):
0
Enter factor:
0
Cannot divide factor by zero!
```

Reflection of Object Oriented Programming Principles

The program uses several *classes* to manage the different possible types of shapes. There are four different kinds of shapes and they all have their own class. However, they all also *inherit* certain values and functions from their *superclass*. For example, the Rectangle class takes its coordinates and number of sides properties from the Shape class. When an object is a Rectangle, it's also a Shape at the same time, with four sides and a certain position.

The Shape class also contains *methods* to get the area and perimeter, but since they are generic shapes these methods are *abstract*. They are implemented specifically in the actual different shape classes.

Appendix: Java Files

Circle.java

```java
public class Circle extends Shape {
  private int radius;

  // construct circle from position and radius. we let a circle have just
one side.
  public Circle(int radius, Coordinates position) {
    super(1, position);
    this.radius = radius;
  }

  // gets area using πr²
  @Override
  double getArea() {
    return Math.PI * radius * radius;
  }

  // gets perimeter using 2πr
  @Override
  double getPerimeter() {
    return 2 * Math.PI * radius;
  }

  // scales coordinates and radius
  @Override
```

```
  public void scale(int factor, boolean sign) {
    super.scale(factor, sign);
    if (sign) this.radius *= factor;
    else this.radius /= factor;
  }

  // displays some info about the circle
  @Override
  String display() {
    return "Circle at: " + this.getCoordinates().display() +
    "\nRadius: " + this.radius +
    "\nArea: " + this.getArea() +
    "\nPerimeter" + this.getPerimeter();
  }
}
```

Coordinates.java

```
/*
 * The Coordinates class is used to store a single whole position on the
cartesian plane.
 * We use it to store the positions of shapes, and the vertices of a
triangles as well.
*/

public class Coordinates {
  private int x;
  private int y;

  // constructor that takes in two integers as the x, y positions.
  public Coordinates(int x, int y) {
    this.x = x;
    this.y = y;
  }

  // get x value
  public int getX() {
    return this.x;
  }

  // get y value
  public int getY() {
    return this.y;
  }

  // get the distance between this point and the point p
  public double distance(Coordinates p) {
    return Math.sqrt((this.getX()-p.getX())*(this.getX()-p.getX())+
(this.getY()-p.getY())*(this.getY()-p.getY()));
  }
```

```
  // translate point by dx in the x-axis and by dy in the y axis
  public void translate(int dx, int dy) {
    this.x += dx;
    this.y += dy;
  }

  // scales point by factor. divides if sign is false
  public void scale(int factor, boolean sign) {
    if (sign) {
      this.x *= factor;
      this.y *= factor;
    } else {
      this.x /= factor;
      this.y *= factor;
    }
  }

  // returns a string displaying information about the point.
  public String display() {
    return "X = " + this.x + ", Y = " + this.y;
  }
}
```

Rectangle.java

```
public class Rectangle extends Shape {
  private int width;
  private int length;

  // constructs rectangle from position, width, and length
  public Rectangle(int width, int length, Coordinates position) {
    super(4, position);
    this.width = width;
    this.length = length;
  }

  // gets area of rectangle (ab)
  @Override
  double getArea() {
    return this.width * this.length;
  }

  // gets perimeter of rectangle (2a+2b)
  @Override
  double getPerimeter() {
    return 2*this.width + 2*this.length;
  }

  // scales rectangle side lengths and positions
```

```
  @Override
  public void scale(int factor, boolean sign) {
    super.scale(factor, sign);
    if (sign) {
      this.width *= factor;
      this.length *= factor;
    }
    else {
      this.width /= factor;
      this.length /= factor;
    }
  }

  // displays information about rectangle
  @Override
  String display() {
    return "Rectangle at: " + this.getCoordinates().display() +
    "\nWidth: " + this.width +
    "\nLength: " + this.length +
    "\nArea: " + this.getArea() +
    "\nPerimeter: " + this.getPerimeter();
  }
}
```

Shape.java

```
/*
 * The shape class is the parent of all shape types of the project. It stores
the position of the object
 * and the number of sides.
 */

abstract class Shape {
  private Coordinates position;
  private int sides;

  // constructor for object. takes the number of sides and the coordinates.
  public Shape(int noOfSides, Coordinates coord) {
    this.position = coord;
    this.sides = noOfSides;
  }

  // gets the coordinates of the shape.
  public Coordinates getCoordinates() {
    return position;
  }

  // gets the number of sides of the shape.
  public int getSides() {
    return sides;
```

```
  }

  // sets the coordinates of the shape.
  public void setCoordinates(Coordinates newcoord) {
    this.position = newcoord;
  }

  // translates shape (by calling Coordinates.translate on its position
aspect)
  public void translate(int dx, int dy) {
    this.position.translate(dx, dy);
  }
  // scales shape position (by calling Coordinates.scale on its position
aspect)
  public void scale(int factor, boolean sign) {
    this.position.scale(factor, sign);
  }

  abstract double getArea(); // will get the area of shape
  abstract double getPerimeter(); // will get perimeter of shape
  abstract String display(); // will display information about the shape
}
```

ShapeList.java

```
import java.util.ArrayList;

public class ShapeList {
  private ArrayList<Shape> listofShapes;

  // initialises shape list using an ArrayList.
  public ShapeList(ArrayList<Shape> shapes) {
    this.listofShapes = shapes;
  }

  // adds shape to list
  public void addShape(Shape s) {
    this.listofShapes.add(s);
  }

  // removes shape at position (while checking for bounds)
  public void removeShape(int pos) {
    if (pos < 0) throw new IndexOutOfBoundsException("Removal index
invalid.");
    else if (pos >= this.listofShapes.size()) throw new
IndexOutOfBoundsException("Removal index invalid.");
    else this.listofShapes.remove(pos);
  }

  // gets shape at position (while checking for bounds)
```

```
  public Shape getShape(int pos) {
    if (pos < 0) throw new IndexOutOfBoundsException("Getting index
invalid.");
    else if (pos >= this.listofShapes.size()) throw new
IndexOutOfBoundsException("Getting index invalid.");
    else return this.listofShapes.get(pos);
  }

  // translates all shapes in list
  public void translate(int dx, int dy) {
    for (Shape shape : listofShapes) {
      shape.translate(dx, dy);
    }
  }

  // scales all shapes in list
  public void scale(int factor, boolean sign) {
    for (Shape shape : listofShapes) {
      shape.scale(factor, sign);
    }
  }

  // gets area of shape at position (while checking for bounds)
  public double area(int pos) {
    if (pos < 0) throw new IndexOutOfBoundsException("Area index invalid.");
    else if (pos >= this.listofShapes.size()) throw new
IndexOutOfBoundsException("Area index invalid.");
    else return this.getShape(pos).getArea();
  }

  // gets perimeter of shape at position (while checking for bounds)
  public double perimeter(int pos) {
    if (pos < 0) throw new IndexOutOfBoundsException("Perimeter index
invalid.");
    else if (pos >= this.listofShapes.size()) throw new
IndexOutOfBoundsException("Perimeter index invalid.");
    else return this.getShape(pos).getPerimeter();
  }

  // gets the length of the list (how many shapes there are)
  public int getNumberOfShapes() {
    return this.listofShapes.size();
  }

  // displays some info about all of the shapes in the shape list
  public String display() {
    String outString = "";
    for (Shape shape : listofShapes) {
      outString += "\n-------\n" + shape.display() + "\n------\n";
    }
    return outString;
```

```
  }
}
```

ShapeManagement.java

```
import java.util.ArrayList;
import java.util.Scanner;



public class ShapeManagement {
  // we have the scanner here as all the menu methods will access it.
  static Scanner userScanner = new Scanner(System.in);
  static ShapeList userShapes = new ShapeList(new ArrayList<>());

  // makes a rectangle from user input
  private static Rectangle makeRectangle() {
    System.out.println("Enter coordinates (x y): ");
    int x = userScanner.nextInt();
    int y = userScanner.nextInt();

    System.out.println("Enter width: ");
    int width = userScanner.nextInt();

    System.out.println("Enter length: ");
    int length = userScanner.nextInt();

    return new Rectangle(width, length, new Coordinates(x, y));
  }

  // makes a square from user input
  private static Square makeSquare() {
    System.out.println("Enter coordinates (x y): ");
    int x = userScanner.nextInt();
    int y = userScanner.nextInt();

    System.out.println("Enter side length: ");
    int side = userScanner.nextInt();

    return new Square(side, new Coordinates(x, y));
  }

  // makes a circle from user input
  private static Circle makeCircle() {
    System.out.println("Enter coordinates (x y): ");
    int x = userScanner.nextInt();
    int y = userScanner.nextInt();

    System.out.println("Enter radius: ");
    int radius = userScanner.nextInt();
```

```
    return new Circle(radius, new Coordinates(x, y));
  }

  // makes a triangle from user input
  private static Triangle makeTriangle() {
    System.out.println("Enter first coordinates (x1 y1): ");
    int x1 = userScanner.nextInt();
    int y1 = userScanner.nextInt();
    Coordinates p1 = new Coordinates(x1, y1);

    System.out.println("Enter second coordinates (x2 y2): ");
    int x2 = userScanner.nextInt();
    int y2 = userScanner.nextInt();
    Coordinates p2 = new Coordinates(x2, y2);

    System.out.println("Enter first coordinates (x3 y3): ");
    int x3 = userScanner.nextInt();
    int y3 = userScanner.nextInt();
    Coordinates p3 = new Coordinates(x3, y3);

    return new Triangle(p1, p2, p3);
  }

  // combine all of the shape makers into a single tool
  private static void makeShape() {
    System.out.println("Enter the type of shape to make:\n\t1:
Rectangle\n\t2: Square\n\t3: Circle\n\t4: Triangle");
    int userChoice = userScanner.nextInt();

    Shape userShape = null;
    if (userChoice == 1) userShape = makeRectangle();
    else if (userChoice == 2) userShape = makeSquare();
    else if (userChoice == 3) userShape = makeCircle();
    else if (userChoice == 4) userShape = makeTriangle();
    else {
      System.out.println("Invalid shape type.");
      return;
    }

    userShapes.addShape(userShape);
  }

  // removes a shape, as long as it is in that position. otherwise print no
shape found.
  private static void removeShape() {
    System.out.println("Enter shape position in list: ");
    int userChoice = userScanner.nextInt();

    try {
      userShapes.removeShape(userChoice);
```

```
    } catch (IndexOutOfBoundsException e) {
      System.out.println("Invalid index.");
    }
  }

  // displays one shape in the user list.
  private static void displayOneShape() {
    System.out.println("Enter shape position in list: ");
    int userChoice = userScanner.nextInt();

    try {
      System.out.println("\n------\n" +
userShapes.getShape(userChoice).display() + "\n------\n");
    } catch (IndexOutOfBoundsException e) {
      System.out.println("Invalid index.");
    }
  }

  // displays area and perimeter information about one shape in the user
list.
  private static void displayOnePerimeterArea() {
    System.out.println("Enter shape position in list: ");
    int userChoice = userScanner.nextInt();

    try {
      Shape userShape = userShapes.getShape(userChoice);
      System.out.println("Area: " + userShape.getArea() + "\nPerimeter: " +
userShape.getPerimeter());
    } catch (IndexOutOfBoundsException e) {
      System.out.println("Invalid index.");
    }
  }

  // displays all shapes in the user list.
  private static void displayShapes() {
    System.out.println(userShapes.display());
  }

  // translates all shapes in the user list.
  private static void translateShapes() {
    System.out.println("Enter x translation: ");
    int dx = userScanner.nextInt();
    System.out.println("Enter y translation: ");
    int dy = userScanner.nextInt();

    userShapes.translate(dx, dy);
  }

  // scales all shapes in the user list.
  private static void scaleShapes() {
```

```
    System.out.println("Divide scale or multiply? (0 for divide, 1 for
multiply): ");
    int userSign = userScanner.nextInt();
    boolean sign = userSign == 1;
    System.out.println("Enter factor: ");
    int factor = userScanner.nextInt();

    try {
      userShapes.scale(factor, sign);
    } catch (ArithmeticException e) {
      System.out.println("Cannot divide factor by zero!");
    }

    return;
  }

  // the main loop of the program
  private static void userLoop() {
    System.out.println(
      "Enter command:\n\t1: Add a shape\n\t2: Remove shape (using
coordinates)\n\t3: Info about one shape (using index)\n\t4: Get perimeter
and area (using coordinates)\n\t5: Display all shapes\n\t6: Translate all
shapes\n\t7: Scale all shapes\n\t0: Quit loop"
    );
    int userInput = userScanner.nextInt();

    while (userInput != 0) {
      switch (userInput) {
        case 1: makeShape(); break;
        case 2: removeShape(); break;
        case 3: displayOneShape(); break;
        case 4: displayOnePerimeterArea(); break;
        case 5: displayShapes(); break;
        case 6: translateShapes(); break;
        case 7: scaleShapes(); break;
        default:
          System.out.println("Unrecognised command. Try again.");
          break;
      }
      System.out.println(
        "Enter command:\n\t1: Add a shape\n\t2: Remove shape\n\t3: Info
about one shape\n\t4: Get perimeter and area\n\t5: Display all shapes\n\t6:
Translate all shapes\n\t7: Scale all shapes\n\t0: Quit loop"
      );
      userInput = userScanner.nextInt();
    }
    userScanner.close();
  }

  public static void main(String[] args) {
    System.out.println("Welcome to shape tool.");
```

```
    userLoop();
    userScanner.close();
  }
}
```

Square.java

```
public class Square extends Shape {
  private int side;

  // construct square with side length and position
  public Square(int side, Coordinates position) {
    super(4, position);
    this.side = side;
  }

  // get area of square (a²)
  @Override
  double getArea() {
    return side * side;
  }

  // get perimeter of square (4a)
  @Override
  double getPerimeter() {
    return 4 * side;
  }

  // scale square: coordinates and side length
  @Override
  public void scale(int factor, boolean sign) {
    super.scale(factor, sign);
    if (sign) this.side *= factor;
    else this.side /= factor;
  }

  // display info about square
  @Override
  String display() {
    return "Square at: " + this.getCoordinates().display() +
    "\nSide length: " + this.side +
    "\nArea: " + this.getArea() +
    "\nPerimeter: " + this.getPerimeter();
  }
}
```

Triangle.java

```
public class Triangle extends Shape {
  private Coordinates p1;
```

```java
  private Coordinates p2;
  private Coordinates p3;

  // construct triangle form points. we let p1 be the coordinate for our
shape.
  public Triangle(Coordinates p1, Coordinates p2, Coordinates p3) {
    this.p1 = p1;
    this.p2 = p2;
    this.p3 = p3;
    super(3, p1);
  }

  // gets perimeter using distance formula between points
  @Override
  double getPerimeter() {
    return p1.distance(p2) + p2.distance(p3) + p3.distance(p1);
  }

  // gets area using heron's formula: √(s(s-a)(s-b)(s-c)) where s =
(a+b+c)/2
  @Override
  double getArea() {
    return Math.sqrt(
      this.getPerimeter()*0.5*
      (this.getPerimeter()*0.5 - p1.distance(p2))*
      (this.getPerimeter()*0.5 - p2.distance(p3))*
      (this.getPerimeter()*0.5 - p3.distance(p1))
    );
  }

  // translates triangle by translating p1, p2, and p3 (as well as the
position of the parent shape properties)
  @Override
  public void translate(int dx, int dy) {
    p1.translate(dx, dy);
    p2.translate(dx, dy);
    p3.translate(dx, dy);
    super.setCoordinates(p1);
  }

  // scales triangle by scaling p1, p2, and p3 (and the position of the
parent shape property)
  @Override
  public void scale(int factor, boolean sign) {
    super.scale(factor, sign);
    p2.scale(factor, sign);
    p3.scale(factor, sign);
  }

  // displays some info about the triangle
  @Override
```

```
  String display() {
    return "Triangle. Vertices:\n\t" +
    p1.display() + "\n\t" + p2.display() + "\n\t" + p3.display() +
    "\nArea: " + this.getArea() +
    "\nPerimeter: " + this.getPerimeter();
  }
}
```